04 从高层目标到技术目标

本册目标：把愿景改写成能设计、能测试的问题

题目关注从处理器架构和软硬件协同出发约束 Agent 自我复制，并支持紧急强制终止。会议补充了多个 Agent 之间的隔离、防止协同作恶。

这些是研究方向，还没有完整说明“谁被控制、控制什么、什么情况下算成功”。本周最有价值的思考，是补上这些空缺，提出一版候选技术目标。

从熟悉的停车场理解

“让停车场安全”太宽泛；“只有获得授权的车牌才能抬杆，紧急停止后不接受新的抬杆请求”更具体。后者能找出身份、检查位置、状态、优先级和测试输入。

Agent 系统也需要这些要素，但“车辆”可能横跨控制程序、工具进程、推理请求、文件和网络行为。先定义这些对象怎样归属同一个受控域，再研究具体机制。

采用一个教学场景

本地有 Agent A 与 Agent B，各有控制任务和工具；它们调用一个共享推理服务 M。受信任管理者 C 设置启动和通信策略，并能够请求停止某个域。

```TEXT
管理者 C --> 启动许可 / 通信策略 / 停止命令
                    |
          +---------+---------+
          v                   v
       Agent A             Agent B
          |                   |
          +------> 服务 M <---+
```

这只是帮助讨论的模型。你最终使用哪种 Agent 框架、CPU 平台与隔离范围，需要在后续架构设计中确定。

> 本周输出一页技术目标草案即可。以下所有具体机制和数值都属于教学候选，不是已实现的毕设结果。

4.1 先写清楚假设

威胁模型：假定对方能做什么

威胁模型是研究的边界说明。可以先假设：Agent 能运行普通用户态代码、调用被授予的工具、尝试创建任务和发送消息；不能直接篡改受信任管理者、内核与受保护的硬件状态。

这只是一个起步版本。如果导师希望应对更强的对手，例如不信任操作系统内核，那么控制点、身份来源和验证方式都要重新审视。

要写清的字段	需要回答的问题	教学候选
保护对象	一个 PID、一个会话，还是一个任务域？	逻辑 Agent 域及其已登记任务
对手能力	能执行什么、修改什么、访问什么？	普通应用与授权工具接口
信任基础	哪些部分必须正确且不能被对手改动？	管理者、内核、受保护控制状态
系统边界	管本机、设备，还是外部机器？	第一阶段只覆盖本机明确的通道
触发来源	谁判断违规、谁发停止命令？	受信任管理者发出命令
暂未覆盖	哪些困难留作后续研究？	所有隐蔽通道、已发生的远端副作用

信任基础也叫 TCB，Trusted Computing Base。它不是越大越好，而是要明确依赖了哪些组件。硬件辅助通常希望减少某些依赖或增强强制执行能力，具体效果需要比较验证。

区分检测与强制执行

检测回答“是否发生了违规”；强制执行回答“即使对方不配合，怎样阻止行为”。本周可把停止触发设为管理者的人工命令，先研究执行路径，避免同时承担所有恶意行为检测问题。

关于 RSI 的处理

可以把更强的自主性、自我改写与复制能力作为威胁设想；实验可使用公开框架或有限的模拟任务。会议纪要关于未公开系统的说法不作为本资料已验证的事实。

练习：把“内核可信”改成“内核可能恶意”，你原先依赖的哪些保护还需要重新论证？

4.2 一套通用的转化方法

用七个字段写一条技术目标

字段	填写方式
对象	明确归属、资源与控制范围
违规行为	用可观察的事件描述，不依赖猜测意图
规则	规定什么允许、什么拒绝、何时切换
控制点	写出在哪里检查、怎样保证不能绕过
假设	列出信任的组件与不覆盖的渠道
指标	写出单位、计时起止点、成功与失败标准
实验	合法对照、违规尝试、边界与并发情况

从一句愿望到一句可测目标

愿望：“Agent 收到紧急命令后必须停止。”

候选技术表述：“在本机、内核与管理者可信的条件下，管理者接受对域 A 的停止命令后，控制器拒绝该域的新启动与发送请求，并等待已登记任务退出；分别记录拒绝开始时间和任务收敛时间，检查是否仍有受控活动。”

它仍需要进一步补充哪些发送接口、哪些任务与设备被纳入，以及需要达到的时间界限。但已经能据此画数据流、加日志、设计失败判据。

好指标的三项特征

可观察：能从受信任日志或检查点得到证据。可复现：别人用相同工作负载和配置能重复实验。与目标对应：测到的是保护效果，而不只是程序是否打印了“停止成功”。

一个明确的失败判据

若控制器已经记录域停止生效，但仍接受该域未经授权的新启动，这就是失败。只看主进程的退出码可能看不到它，因此观测点必须覆盖规则涉及的对象。

练习：把“防止共谋”改写成一句只涉及共享文件与本地套接字、能被实验反驳的目标。避免使用“绝对安全”之类无法界定范围的词。

4.3 目标一：可靠停止

把停止拆成阶段，分别计时

时间点	具体定义	可观察证据
t0	受信任控制器接受停止命令	命令接收日志
t1	域的新请求开始被拒绝	控制点的拒绝记录
t2	声明范围内的任务与在途操作收敛	任务状态及受控副作用检查

拒绝新请求的延迟是 t1-t0；收敛延迟是 t2-t0。这里没有包括“发现恶意行为所用时间”，因为触发来源在本实验里是人工命令。t2 必须先定义范围；若不包含远端请求，就不能用它宣称远端也已停止。

建议的候选指标

- 停止后受控新启动与新发送的接受次数应为 0。
- 声明范围内的活动任务数收敛到 0；明确观察窗口。
- 重复实验记录延迟分布、最大观测值、负载与失败次数。
- 同时运行的合法域 B 是否继续工作，作为误伤检查。

如果要写“最多若干毫秒”，应先有模型或实验依据，再说明平台、负载和假设。多次测试得到的最大观测值，并不自动构成所有情况下的严格上界。

从 RTL 示例到系统实验

停止门可以检查“采样到命令后，下一个状态为停止，allow 为 0”。系统还要处理域中任务、设备队列、打开连接和已发请求。时钟周期数与操作系统完成终止的毫秒数不能直接混成同一个指标。

四项必须覆盖的测试

正常任务中途停止；停止时恰有新任务创建；重复发送停止命令；停止与恢复命令同时到来。分别规定优先级与预期结果。

选读：测短时延可以用 Python `time.perf_counter()` 的时间差；若跨设备或机器计时，还需要考虑时钟基准与同步问题。[P2]

4.4 目标二：约束自我复制

先定义“复制成功”是什么

模型文件复制一份、创建一个普通工具进程、启动一个具备 Agent 行动能力的新实例、把任务状态移交到其他机器，可能属于不同事件。保护规则要围绕你的工作定义设计。

一个可讨论的候选是：“未经受信任管理者授权，不得在受控边界内激活一个具有独立行动能力的新 Agent 实例；所有合法派生任务应继承可追踪的域归属。”

为什么不直接禁止所有子进程

很多合法工具本身需要子进程。完全禁止可能破坏正常工作流。需要区分普通工具任务与新的 Agent 实例，或采用统一任务域管理，再为可激活能力、凭证和启动操作设置规则。

可能的路径	需要检查的问题
创建新任务	任务是否被绑定到正确域，是否超出配额？
在已有进程中激活新会话	没有新增 PID 时如何登记新实例？
保存状态后恢复	恢复时是否重新验证身份与授权？
调用外部执行服务	谁持有凭证，是否属于受控出口？

候选实验

授权启动一个实例应成功；未授权启动应拒绝；停止后即使旧授权存在也应按规范拒绝；多个并发请求不得绕过配额；合法工具任务仍能正常完成。

可记录“未授权激活成功数”“合法启动误拒次数”“域归属遗漏数”和性能开销。有限测试下未授权成功数为 0，是该测试集合的结果，不是对任意未知路径的数学证明。

本周草案要诚实保留的难点

如何识别一个通用程序已经成为新 Agent，可能很难。可以先限制实验框架，要求新会话与任务经过明确的激活接口，再讨论绕过该接口的可能性。这一假设必须写进结论。

练习：进程数没有增加，是否足以证明没有出现新的 Agent 实例？给一个场景。

4.5 目标三：限制跨 Agent 信息流

从“不能共谋”收敛到明确的通信规则

单靠观察输出，很难证明两个智能体绝对没有协同意图。可以先研究更具体的系统目标：在声明的通道集合上，未经授权的信息传递无法被接受。

例如规定 A 和 B 不能直接通信，但都可以访问推理服务 M 的独立会话：

发送方 → 接收方	A	B	M
A	本域内允许	拒绝	按独立会话授权
B	拒绝	本域内允许	按独立会话授权
M	仅返回 A 的响应	仅返回 B 的响应	服务内部规则

允许 A→M 和 B→M 不代表自动隔离。共享服务若把 A 的提示、缓存或文件暴露给 B，仍可能成为间接通道。需要明确服务本身是否可信、哪些状态可共享。

测试按渠道组织

- 共享文件：A 写入测试消息，B 在未授权目录或文件上读取应失败。
- 本地套接字：跨域端点连接或消息投递应被拒绝。
- 共享内存：未授权跨域映射应被拒绝。
- 网络：若纳入范围，就测试直接连接与已声明的中转路径。
- 合法对照：同域通信、授权服务返回应能工作。

结果应该怎样写

“在列出的接口、配置和测试工作负载下，未授权消息投递成功数为 0；授权通信正常完成。隐蔽通道和未纳入的外部服务仍需单独研究。”

性能代价也值得记录，例如通信延迟增加多少、正常任务吞吐下降多少。性能属于保护机制的代价评估，本课题的主要安全目标仍是阻止声明范围内的恶意行为。

练习：若所有本地 IPC 被隔离，但 A、B 都能访问同一个外部消息服务，你应如何限定结论？

4.6 软硬件协同从哪里入手

先画功能分工，不急着确定最终架构

```TEXT
受信任管理者
  | 配置身份、授权关系、停止 / 恢复
  v
受保护状态：域标识、权限、停止位、必要计数
  |                        |
  v                        v
软件接口检查           候选硬件检查点
启动 / IPC / I/O      请求归属 / 权限 / 停止状态
  |                        |
  +----------+----------+
             v
      执行结果、拒绝与完成确认
```

这是功能示意图。是否需要处理器指令扩展、内存标签、总线检查或设备配合，必须在威胁模型、平台与已有机制比较之后决定。

一个适合初学者思考的竞态

事件 1：创建任务的线程检查“域仍可运行”。事件 2：另一个线程把域设为停止。事件 3：创建线程沿用旧检查结果，提交新任务。

这叫检查与使用之间的竞态，也常简称 TOCTOU。不能只说“启动前检查一次就安全”。应明确生效点，以及检查、任务归属绑定、提交与停止之间如何排序或协调。

本周可以提出的最小原型

先做停止位与请求授权门的 RTL 模型；用软件逻辑模型表示域和通信；下一步再选择实际系统接口，把策略连接到真实执行路径。各阶段验证的问题不同，要分别报告。

比较对象

后续研究可比较“单任务停止”“已有组管理机制”“加入所提硬件辅助机制”，保持工作负载和信任假设可比。需要说明新增机制具体补足了什么保证，以及代价是什么。

练习：如果硬件只看一个由恶意应用自行填写的域编号，它能可靠知道请求属于谁吗？还缺什么？

4.7 动手：运行一个逻辑策略模型

用最少代码验证规则是否自洽

配套 policy_model.py 用一个集合记录活动实例、一个集合记录允许的通信边，以及一个 stopped 标志记录停止状态。集合是保存不重复成员的数据结构。

```python
domain = Domain()
assert not domain.start("B")
assert domain.start("B", authorized=True)
assert not domain.send("A", "B")
domain.edges.add(("A", "B"))
assert domain.send("A", "B")
assert not domain.send("B", "A")
domain.stop()
assert not domain.active
assert not domain.start("C", authorized=True)
assert not domain.send("A", "B")
```

运行命令：

```text
python3 policy_model.py
```

此命令用于 macOS / Ubuntu；Windows PowerShell 用 py -3 policy_model.py。也可运行 05 中自己系统的 Python 入口。 预期输出：PASS policy_model: 8 checks。assert 是断言，条件不成立时测试会失败；not 表示取反。start 返回逻辑上是否接受启动，send 返回是否允许发送，均不创建真实进程或网络消息。

你能从中学到什么

没有授权时启动被拒绝；授权边是有方向的，允许 A→B 不等于允许 B→A；停止后模型拒绝新启动与通信。

你不能据此证明什么

模型清空活动集合，不会终止操作系统中的任务。任意程序也能绕开这个 Python 对象直接操作系统。因此这只是规则演示，不能被描述成已实现的沙箱、硬件保护或 Agent 停止系统。

修改练习

不运行 stop，先添加 B→A 的授权边，再预测反向 send 的返回值。然后思考：如果要增加“最多两个活动实例”的规则，还需要处理释放、并发和重复名字中的哪些情况？

4.8 你可以填写的一页技术目标草案

建议带到下一次组会讨论

研究边界：第一阶段针对本机的受控 Agent 框架。管理者与内核暂定可信；实例激活与选定通信操作需要经过声明的控制接口。远端自主执行和所有隐蔽通道暂未纳入。

高层目标	候选技术目标	最小验证方式
硬停止	接受停止命令后拒绝域的新启动和发送，并跟踪已登记任务收敛	记录 t0/t1/t2；停止期间并发创建；检查残留及对其他域的影响
限制复制	未授权的新实例激活被拒绝，合法派生任务具有可追踪归属	授权 / 未授权对照；停止后启动；配额并发与恢复测试
多 Agent 隔离	选定的文件、共享内存与套接字渠道遵守跨域授权矩阵	未授权消息尝试；合法通信对照；共享服务状态检查

需要继续填写的空白

```text
受控 Agent 实体的工作定义：
所用 Agent 框架与运行平台：
域身份如何绑定到任务 / 会话：
纳入范围的具体接口与设备：
停止完成的判定与观察窗口：
预期时间指标及其依据：
软件基线能提供的保证：
拟由硬件增强的那一项保证：
当前最不确定的问题：
```

优先问导师的三个问题

1. 会议中的工具原名是否为 Chisel，预期使用哪套开发平台？
2. 第一阶段默认操作系统内核可信吗？管理边界是否先限定本机？
3. 毕设阶段希望优先实现哪一个目标，其他目标做到怎样的讨论或验证深度？

这些问题用于收敛后续研究范围。你本周可以先按明确写出的候选假设完成学习与草案，不需要等待全部架构细节确定。

4.9 综合自测与参考答案

先口头回答，再核对

1. 给停止目标加一个时间指标，至少要定义哪两个东西？
2. “复制权重文件”与“激活一个新 Agent”为什么不等价？
3. 为什么隔离目标要列出渠道，而不能只说“不同进程”？
4. 控制器打印“已停止”，是否足以验证所有任务停止？
5. 策略模型 8 条断言通过，能否证明真实操作系统不能被绕过？
6. 停止与创建并发时，为什么只在开始检查一次可能失败？
7. 第一阶段默认内核可信，后来增加硬件检查点，是否就自动不再信任内核？
8. 五天后的技术目标草案至少应包含哪些内容？

参考答案

1. 定义计时起止点与观察范围，还要说明平台、负载、成功失败判据。
2. 权重是数据，行动能力还依赖运行逻辑、状态、工具与授权；新会话也可能共享权重。
3. 进程可经文件、共享内存、套接字等传递信息，不同地址空间不能覆盖这些路径。
4. 不足。需要在可信观测点检查声明范围内的活动、在途操作和停止后的新请求。
5. 不能。模型验证规则，系统保护还要求真实接口上的强制执行与覆盖。
6. 检查后状态可能改变；提交时若沿用过期结论，就可能在停止后创建任务。
7. 不自动。需重新分析身份、配置、异常处理与设备路径还依赖哪些软件。
8. 对象、威胁与信任假设、规则、控制点、指标、实验，以及待导师明确的问题。

你应能说出的三句话

“我知道要保护哪个域、域里有哪些对象。”

“我能用事件和接口描述允许与拒绝的规则。”

“我能设计一个可能让这条规则失败的实验，并说明通过测试后结论能覆盖到哪里。”

阅读来源

[P2] Python 时间测量：https://docs.python.org/3/library/time.html

[05] 既有组管理机制：https://docs.kernel.org/admin-guide/cgroup-v2.html

本册的目标拆分、场景、策略模型与草案为教学设计，研究方向来自本地题目说明与 11.6 会议纪要。